

GitHub Copilot Hands-On Exercises

What You'll Build

In these exercises, you'll build a **simple C# console application** with three useful components:

- A **Calculator** for basic arithmetic operations
- A **Task Manager** to track and manage tasks
- A **Data Processor** to analyze collections of numbers

Through building this application, you'll learn to leverage GitHub Copilot's powerful features to write code faster and more efficiently.

Overview

This folder contains a simple C# console application designed to help you practice using GitHub Copilot's three main features:

1. **Inline Suggestions** - Code completions as you type
2. **Chat Ask Mode** - Ask questions and get explanations
3. **Chat Agent Mode** - Let Copilot make changes to your code

Prerequisites

- Visual Studio 2022 or Visual Studio Code installed
- GitHub Copilot extension installed and activated
- .NET 8.0 SDK or later installed

Getting Started

Opening the Project

In Visual Studio:

1. Open Visual Studio 2022
2. Click "Open a project or solution"
3. Navigate to this folder and select **CopilotExercises.csproj**
4. The project will have build errors initially - this is expected for Exercise 1

In VS Code:

1. Open VS Code
2. File → Open Folder
3. Select the **copilot-exercises** folder
4. Open the integrated terminal
5. Run **dotnet build** - you'll see build errors until Exercise 1 is complete

Exercise 1: Inline Suggestions (Copilot Completions)

Objective: Learn how to use GitHub Copilot's inline code suggestions to write code faster.

Files to Edit:

- `Exercise1-InlineSuggestions/Calculator.cs`

Instructions:

1. **Open** `Exercise1-InlineSuggestions/Calculator.cs` in your IDE

2. **Understand the starting point:**

- The Calculator class only has TODO comments
- No methods are implemented yet
- The project will have build errors until you add the methods
- This is expected and part of the learning process!

3. **Implement the Add method:**

- Position your cursor after the comment `// TODO: Implement Add method...`
- Press Enter to create a new line
- Start typing: `public int Add(`
- Watch as Copilot suggests the complete method signature and implementation
- Press **Tab** to accept the suggestion
- If the suggestion isn't what you want, press **Esc** and try typing more context

4. **Implement the Subtract method:**

- Move to the Subtract TODO comment
- Start typing: `public int Subtract(`
- Accept Copilot's suggestion with Tab

5. **Implement the Multiply method:**

- Try typing just: `public int`
- Notice how Copilot might predict you want a Multiply method based on context
- Accept or modify the suggestion as needed

6. **Implement the Divide method:**

- This one requires error handling for division by zero
- Start typing: `public double Divide(`
- See if Copilot suggests error handling
- If not, add a comment above: `// Handle division by zero`
- Then try implementing the method again

Tips:

- Copilot learns from comments - writing descriptive comments before methods can improve suggestions
- The project won't build until you implement all methods - this is normal for Exercise 1
- Try different approaches - type partial signatures and see what Copilot suggests

Success Criteria:

- All four methods are implemented
- The Calculator class has no compilation errors (the project builds successfully)
- Division by zero is handled properly

Testing Your Implementation:

Important: The project will not build until you implement all four methods in Calculator.cs.

Once all methods are implemented, run the program to see automated test results:

- Visual Studio: Press F5
- VS Code Terminal: `dotnet run`

The test script will show you which methods are implemented correctly with color-coded output.

You can also run `dotnet build` to check if your code compiles before running the tests.

Exercise 2: Chat Ask Mode

Objective: Learn to use Copilot Chat to ask questions and get explanations without modifying code.

File to Reference: `Exercise2-ChatAskMode/TaskManager.cs`

Instructions:

1. Open the Copilot Chat Panel

- Visual Studio: Ctrl+Q → Search "GitHub Copilot" → Open panel
- VS Code: Ctrl+Alt+I or click the chat icon

2. Toggle to Ask Mode

- Ensure you're in Ask Mode (not Agent Mode) so Copilot won't change your code

3. Ask about implementation approaches:

Prompt 1: Ask Copilot how to implement the AddTask method

```
How should I implement the AddTask method in the TaskManager class?
```

- Read the explanation
- Notice that Copilot provides code suggestions but doesn't modify your files

4. Get explanations about C# concepts:

Prompt 2: Learn about List operations

```
What are the different ways to remove an item from a List<string> in C#?
```

- Review the different approaches Copilot suggests

5. Ask for best practices:

Prompt 3:

```
What's the best way to handle the case where I try to remove a task that doesn't exist?
```

- Consider the advice for error handling

6. Request code explanations:

- Highlight the `private List<string> tasks` line
- Right-click → "Copilot" → "Explain This" (or use chat)
- **Prompt 4:**

```
@workspace Explain what #selection does and why it's declared as private
```

7. Now implement the methods yourself using the knowledge you gained:

- Open `Exercise2-ChatAskMode/TaskManager.cs`
- Each method has a signature but throws `NotImplementedException`
- Delete the `throw` statements and implement each method:
 - Implement `AddTask` method
 - Implement `RemoveTask` method
 - Implement `ListTasks` method
 - Implement `GetTaskCount` method

Tips:

- Use `@workspace` to reference your entire workspace
- Use `#selection` or `#file` to reference specific code
- Ask follow-up questions to get more details
- Ask Mode doesn't change your code - you stay in control

Success Criteria:

- You've asked at least 3 questions in Chat
 - You understand the suggested implementations
 - All methods in `TaskManager` are implemented (no more `NotImplementedException`)
 - No compilation errors
 - When you run `dotnet run`, Exercise 2 shows results instead of yellow warnings
-

Exercise 3: Chat Agent Mode

Objective: Learn to use Copilot in Agent Mode to let it make direct changes to your code.

File to Modify: `Exercise3-ChatAgentMode/DataProcessor.cs`

Note: Like the previous exercises, the methods already exist with signatures but throw `NotImplementedException`. You'll use Copilot Agent Mode to replace these with actual implementations.

Instructions:

1. **Open** `Exercise3-ChatAgentMode/DataProcessor.cs`
2. **Open the Copilot Chat Panel** (if not already open)
3. **Use Agent Mode to implement multiple methods:**

Prompt 1: Implement the Sum method

Replace the `NotImplementedException` in the `Sum` method in `Exercise3-ChatAgentMode/DataProcessor.cs` with code that calculates the sum of a list of integers

- Copilot will analyze your file and suggest changes
- Review the suggested code
- Click "Accept" to apply the changes directly to your file
- Notice how Agent Mode modifies your code automatically

4. **Implement Average with specific requirements:**

Prompt 2:

Replace the `NotImplementedException` in the `Average` method in `Exercise3-ChatAgentMode/DataProcessor.cs`. It should return 0 if the list is empty to avoid division by zero.

- Review and accept the changes
- Notice how Copilot includes the error handling you specified

5. **Implement multiple methods at once:**

Prompt 3:

Replace the `NotImplementedException` in both `FindMax` and `FindMin` methods in `Exercise3-ChatAgentMode/DataProcessor.cs`. They should handle empty lists gracefully by returning 0.

- Copilot should implement both methods
- Review each change carefully before accepting

6. Refactor with Agent Mode:

Prompt 4:

Replace the `NotImplementedException` in the `FilterEvenNumbers` method in `Exercise3-ChatAgentMode/DataProcessor.cs` with a LINQ implementation for a clean, functional approach

- Accept the implementation
- Notice how you can guide the implementation style

7. Request improvements:

Prompt 5:

Add XML documentation comments to all public methods in `Exercise3-ChatAgentMode/DataProcessor.cs`

- Agent Mode can enhance existing code with documentation

Advanced Agent Mode Tasks:

8. Try these additional prompts:

Prompt 6: Add error handling

Add null checking to all methods in `Exercise3-ChatAgentMode/DataProcessor.cs` and throw `ArgumentNullException` when appropriate

Prompt 7: Add unit tests

Create a new file `Exercise3-ChatAgentMode/DataProcessorTests.cs` with unit tests for the `DataProcessor` class

Tips:

- Always review changes before accepting them
- You can ask Copilot to undo or modify suggested changes
- Be specific about requirements (error handling, LINQ, patterns, etc.)
- Agent Mode is powerful but you're still in control - you can reject suggestions

Success Criteria:

- All methods in `DataProcessor` are implemented using Agent Mode (no more `NotImplementedException`)
 - Methods include proper error handling
 - You understand the difference between Ask Mode and Agent Mode
 - Code compiles and runs successfully
 - When you run `dotnet run`, Exercise 3 shows results instead of yellow warnings
-

Exercise 4: Running and Testing Your Code

Instructions:

1. Build the project:

- Visual Studio: Press [Ctrl] + [B] or Build → Build Solution
- VS Code Terminal: `dotnet build`

2. Run the application:

- Visual Studio: Press F5 or Debug → Start Debugging
- VS Code Terminal: `dotnet run`

3. Verify output:

- You should see output from all three classes
- Calculator operations should show correct results
- Task Manager should list tasks
- Data Processor should show sum and average

4. Use Copilot to debug issues:

- If you see any errors or unexpected output, use Chat Ask Mode:

Why is my `Calculator.Divide` method throwing an exception?

or

The average calculation seems wrong. Can you help me debug it?

Bonus Challenges

Challenge 1: Expand the Calculator

Prompt:

Add methods to Exercise1-InlineSuggestions/Calculator.cs for Power (exponentiation) and SquareRoot operations. Include appropriate error handling.

Challenge 2: Enhance Task Manager

Prompt:

Refactor Exercise2-ChatAskMode/TaskManager.cs to use a Task class with properties for Name, DueDate, and IsCompleted. Update all methods accordingly.

Challenge 3: Add Data Validation

Prompt:

Create a new class DataValidator.cs with methods to validate email addresses, phone numbers, and URLs using regex.

Challenge 4: Generate Documentation

Prompt:

Generate a README.md file that documents all the classes and methods in this project with usage examples.

Key Takeaways

Inline Suggestions

- ✨ Fast code completion as you type
- 🗣️ Use Tab to accept, Esc to dismiss
- 💡 Comments guide better suggestions

Chat Ask Mode

- ? Ask questions without changing code
- 📖 Get explanations and best practices
- 🔗 Use @workspace and #file references
- 🧠 Learn before implementing

Chat Agent Mode

- 🤖 Copilot makes direct code changes
- ⚡ Implement multiple features quickly

- 🧠 Specify implementation style and requirements
 - ☒ Always review before accepting
-

Next Steps

1. **Practice Daily:** Use Copilot in your real projects
2. **Experiment:** Try different prompting styles to see what works best
3. **Stay Curious:** Ask Copilot to explain unfamiliar code patterns
4. **Share Knowledge:** Help teammates learn effective Copilot usage

Troubleshooting

Copilot not showing suggestions?

- Check that you're signed in to GitHub
- Verify your Copilot subscription is active
- Try restarting your IDE

Suggestions are not relevant?

- Add more context in comments
- Type more of the method signature
- Try rephrasing your intent

Agent Mode not working?

- Ensure you're using Chat (not just inline suggestions)
 - Be specific in your prompts
 - Reference specific files or code sections
-

Additional Resources

- [GitHub Copilot Documentation](#)
- [Copilot Best Practices](#)
- [Getting Started Guide](#)

Happy Coding with Copilot! 🚀